

基于 llama.cpp 的从算法实现到手机端部署的完整方案

0. 项目目标

本项目目标是在 llama.cpp 中支持一种基于 Monarch 矩阵的结构化模型压缩方案，并最终完成 Windows / WSL / Android 手机端部署验证。

与普通低比特量化不同，本方案不是简单将 dense weight 量化为 INT4 / INT8，也不是使用普通非结构化稀疏格式，而是将 LLaMA 模型中的部分 Linear 权重近似表示为 Monarch 矩阵：

$$W \approx P_L \cdot L \cdot P_R \cdot R$$

其中：

P_L / P_R ：置换矩阵或置换索引

L / R ：分块对角矩阵

在 $n = m^2$ 的 square matrix 情况下，Monarch 矩阵可以将原始 dense 矩阵的参数量从：

$$O(n^2)$$

降低到约：

$$O(n^{\{3/2\}})$$

对于 LLaMA-2-7B 中的 4096×4096 attention projection，取：

block_size = 64

num_blocks = 64

则每个 Monarch Linear 只需要保存：

L: [64, 64, 64]

R: [64, 64, 64]

perm: [4096]

而不是完整的：

W: [4096, 4096]

从而实现计算量降低，并进一步获取推理加速。

1. 整体流程概览

完整流程分为五个阶段（以LLaMA-2-7B为例）：

阶段 1: 拟合 Monarch 参数

阶段 2: 展开/对照并验证模型质量

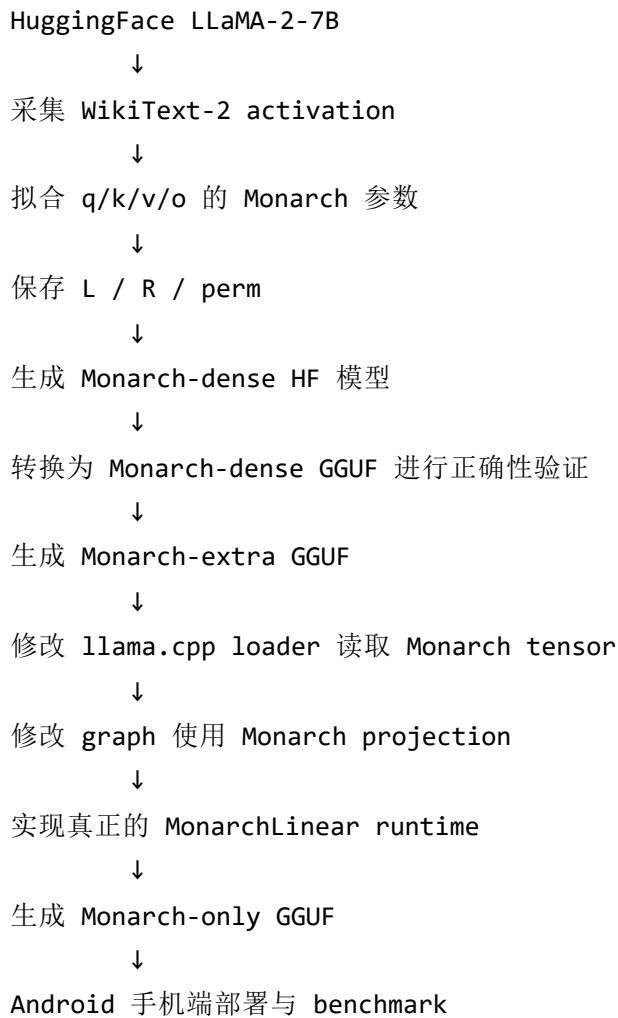
阶段 3: 生成 Monarch-extra GGUF

阶段 4: 修改 llama.cpp loader / graph / runtime

阶段 5: Windows + Android 部署与 benchmark

- 注意，由于实验环境限制，本方案实现时采用了WSL+windows端的本地化部署，如果采用全 linux 端的方案需要相应适当调整。

整体路径为：



2. 推荐目录结构

建议在 WSL 中建立如下目录：

```
~/monarch_llama/
├─ llama.cpp-baseline/      # 原始 llama.cpp, 不改, 用来对照
├─ llama.cpp-monarch/      # 修改后的 llama.cpp, 用于 Monarch 实验
├─ models/
│   ├─ llama-2-7b-hf/       # 原始 HuggingFace 模型
│   └─ llama-2-7b-hf-monarch-dense/
├─ monarch_params/
│   └─ llama2_7b_wikitext2/ # PyTorch 拟合得到的 Monarch 参数
├─ gguf_outputs/
├─ logs/
└─ scripts/
```

其中：

llama.cpp-baseline/

用于原始模型转换、原始模型 benchmark、普通量化模型对照。

llama.cpp-monarch/

用于修改 converter、loader、graph、ggml 算子和 Android 端编译。

3. PyTorch 侧 Monarch 参数生成

3.1 目标层

当前优先处理 LLaMA-2-7B 中所有 4096×4096 的 attention projection:

```
model.layers.i.self_attn.q_proj
model.layers.i.self_attn.k_proj
model.layers.i.self_attn.v_proj
model.layers.i.self_attn.o_proj
```

3.2 拟合目标

对每个目标 Linear 层，采集真实输入 activation:

X = input activation of Linear layer

拟合目标为:

$\text{MonarchLinear}(X) \approx \text{DenseLinear}(X)$

而不是仅拟合:

$W_{\text{monarch}} \approx W_{\text{dense}}$

这样可以让 Monarch 近似更贴近真实推理分布。

3.3 输出文件

每个目标层保存一个 .pt 文件：

```
model__layers__0__self_attn__q_proj.pt
model__layers__0__self_attn__k_proj.pt
model__layers__0__self_attn__v_proj.pt
model__layers__0__self_attn__o_proj.pt
...
```

每个文件包含：

```
{
  "layer_name": "model.layers.0.self_attn.q_proj",
  "in_features": 4096,
  "out_features": 4096,
  "block_size": 64,
  "num_blocks": 64,
  "L": Tensor[64, 64, 64],
  "R": Tensor[64, 64, 64],
  "perm": Tensor[4096],
  "metrics": {
    "mse": ...,
    "rel_mse": ...,
    "mae": ...,
    "rel_mae": ...
  }
}
```

4. Monarch-dense HF 模型验证

4.1 目的

这一步将 Monarch 参数重新展开为 dense weight，替换原始 HuggingFace 模型中的 q/k/v/o 权重。

该步骤的目的不是压缩，而是验证：

1. 拟合得到的 **Monarch** 参数是否合理；
2. 展开后的权重是否能让模型正常生成；
3. 模型输出是否明显崩坏；
4. 后续 `llama.cpp` 修改前先排除拟合质量问题。

4.2 重构公式

PyTorch 中的 Monarch forward 为：

```
x1 = x @ R_blockdiag
x2 = x1[:, perm]
y  = x2 @ L_blockdiag
```

普通 Linear 为：

$$y = x @ W^T$$

因此重构：

$$\begin{aligned} W^T &= R_dense @ P @ L_dense \\ W &= (R_dense @ P @ L_dense)^T \end{aligned}$$

4.3 输出路径

`~/monarch_llama/models/llama-2-7b-hf-monarch-dense/`

4.4 转换为 GGUF

```
cd ~/monarch_llama/llama.cpp-baseline

python convert_hf_to_gguf.py \
  ~/monarch_llama/models/llama-2-7b-hf-monarch-dense \
  --outfile ~/monarch_llama/gguf_outputs/llama2-7b-monarch-dense-f16.gguf \
  --outtype f16
```

4.5 验证运行

```
./build/bin/llama-cli \
  -m ~/monarch_llama/gguf_outputs/llama2-7b-monarch-dense-f16.gguf \
  -p "The future of on-device large language models is" \
  -n 64
```

如果能正常生成，说明 Monarch 参数至少可以支撑模型运行。

5. Monarch-extra GGUF 生成

该文件用于调试 llama.cpp 的 loader 和 graph 修改。

5.1 修改 convert_hf_to_gguf.py

复制原始脚本：

```
cd ~/monarch_llama/llama.cpp-monarch

cp convert_hf_to_gguf.py convert_hf_to_gguf_monarch.py
```

在 convert_hf_to_gguf_monarch.py 中增加：

```
--monarch-dir  
--monarch-dtype
```

其中：

```
--monarch-dir
```

指向 PyTorch 保存的 .pt 参数目录。

```
--monarch-dtype
```

控制 L/R 写入 GGUF 的类型。

5.2 HF 层名到 GGUF tensor 名映射

```
model.layers.0.self_attn.q_proj → blk.0.attn_q  
model.layers.0.self_attn.k_proj → blk.0.attn_k  
model.layers.0.self_attn.v_proj → blk.0.attn_v  
model.layers.0.self_attn.o_proj → blk.0.attn_output
```

新增 tensor：

```
blk.0.attn_q.monarch_l  
blk.0.attn_q.monarch_r  
blk.0.attn_q.monarch_perm
```

5.3 Monarch 参数包输入校验

convert_hf_to_gguf_monarch.py 在向 GGUF writer 注册 extra tensor 前，会先校验参数包。该校验属于转换前的快速失败保护，不改变 --monarch-dir、--monarch-dtype、GGUF tensor 名称或 dense weight 保留策略。

当前可确认的检查包括：

1. .pt 使用 torch.load(..., weights_only=True) 安全读取，也支持无需 PyTorch 的 .npz 参数包；

2. 保存对象必须是 mapping，并包含字符串类型的 `layer_name`（或兼容字段 `target_module_name`）以及 `L`、`R`、`perm`；
3. `L/R` 必须是浮点、有限值、形状一致的三维 tensor，单个 block 必须为方阵；
4. `perm` 必须是一维整数 tensor，长度等于 `num_blocks * block_size`，并且恰好包含 `[0, n)` 的每个索引一次；
5. 根据 GGUF architecture 执行 projection 白名单：LLaMA 允许 attention `q_proj/k_proj/v_proj/o_proj`，Qwen2 当前只允许方阵 `q_proj/o_proj`；
6. 两个文件映射到同一个 GGUF base tensor 时直接报错；
7. 参数目录中没有任何受支持的 attention 层时直接报错，避免生成一个名义上启用 Monarch、实际未写入 extra tensor 的文件。

相关实现：

- `llama.cpp-monarch/monarch_tensor_validation.py`
- `llama.cpp-monarch/convert_hf_to_gguf_monarch.py`

本地回归测试：

```
python -m unittest discover \
-s llama.cpp-monarch/tests \
-p "test_monarch_tensor_validation.py" \
-v
```

这可以保证转换后的参数能被合法读入，并在后续实际删去原有权重，仅保留优化后 MONET 的权重。

5.4 生成命令

根据原有权重转换出 MONET 所需权重

```
cd ~/monarch_llama/llama.cpp-monarch

python convert_hf_to_gguf_monarch.py \
~/monarch_llama/models/llama-2-7b-hf \
--outfile ~/monarch_llama/gguf_outputs/llama2-7b-monarch-extra.gguf \
--outtype f16 \
--monarch-dir ~/monarch_llama/monarch_params/llama2_7b_wikitext2 \
--monarch-dtype f32
```

5.5 检查 GGUF

```
python gguf-py/examples/reader.py \  
~/monarch_llama/gguf_outputs/llama2-7b-monarch-extra.gguf \  
> ~/monarch_llama/logs/monarch_extra_gguf_dump.txt
```

检查 Monarch tensor 数量：

```
grep "monarch_" ~/monarch_llama/logs/monarch_extra_gguf_dump.txt | wc -l
```

如果输出：

```
384
```

说明 128 个 attention projection 的 Monarch 参数都已写入 GGUF。

6. llama.cpp 源码修改总览

下面是从当前已经修改(或者如果进行拓展需要参考修改)的完整源码修改路线。

```
convert_hf_to_gguf_monarch.py  
src/llama-model.cpp  
src/llama-model.h  
src/models/llama.cpp  
src/models/llama.cpp  
src/llama-graph.cpp / src/llama-graph.h  
src/llama-model.h  
ggml/include/ggml.h  
ggml/src/ggml.c 或 ggml/src/ggml.cpp  
ggml/src/ggml-cpu/ggml-cpu.c （或对应 CPU backend 文件）  
ggml/src/ggml-cpu/ggml-cpu.cpp  
CMakeLists.txt （或 backend 注册文件）
```

具体文件名会随 llama.cpp 版本变化，但逻辑模块不变：

1. 模型 loader
2. graph 构建
3. ggml op 定义
4. CPU backend 实现
5. Android 编译同步

7. 修改 src/llama-model.h

7.1 目的

在每层 llama_layer 中保存 Monarch tensor 指针。

7.2 新增结构

在 struct llama_layer 前面加入：

```
struct llama_monarch_weight {  
    ggml_tensor * l      = nullptr;  
    ggml_tensor * r      = nullptr;  
    ggml_tensor * perm   = nullptr;  
  
    bool enabled = false;  
  
    int32_t block_size = 0;  
    int32_t num_blocks = 0;  
};
```

7.3 修改 struct llama_layer

在 attention projection 权重附近加入：

```
llama_monarch_weight wq_monarch;  
llama_monarch_weight wk_monarch;  
llama_monarch_weight wv_monarch;  
llama_monarch_weight wo_monarch;
```

对应：

```
wq_monarch → q_proj  
wk_monarch → k_proj  
wv_monarch → v_proj  
wo_monarch → o_proj
```

四组字段是公共层级存储，不表示每种架构都会启用四组 factor。当前策略为：

架构	当前覆盖 projection	说明
LLaMA	q/k/v/o 中实际为方阵且存在完整三元组的项	-
Qwen2	q/o	Qwen2.5 GQA 的 K/V 为矩形，当前保持 dense；MLP 同样保持 dense

8. 修改 src/models/llama.cpp：loader 阶段

8.1 当前 LLaMA tensor 加载位置

在 load_arch_tensors() 中，当前 LLaMA 创建 attention tensor 的代码为：

```
create_tensor_qkv(layer, i, n_embd, n_embd_head_k * n_head, n_embd_k_gqa, n_embd_v_gqa, 0);  
layer.wo = create_tensor(tn(LLM_TENSOR_ATTN_OUT, "weight", i), {n_embd_head_k * n_head, n_embd
```

其中：

```
create_tensor_qkv(...) 创建 q/k/v  
layer.wo = create_tensor(...) 创建 o_proj
```

8.2 插入 Monarch tensor 认领逻辑

当前 LLaMA loader 在创建 dense q/k/v/o 后调用公共 helper:

```
create_tensor_monarch(layer.wq_monarch, i, LLM_TENSOR_ATTN_Q, "attn_q", layer.wq);
create_tensor_monarch(layer.wk_monarch, i, LLM_TENSOR_ATTN_K, "attn_k", layer.wk);
create_tensor_monarch(layer.wv_monarch, i, LLM_TENSOR_ATTN_V, "attn_v", layer.wv);
create_tensor_monarch(layer.wo_monarch, i, LLM_TENSOR_ATTN_OUT, "attn_output", layer.wo);
```

公共 helper 按每个 dense tensor 的实际 shape 判断, 不再统一假定 q/k/v/o 都是 $n_embd \times n_embd$ 。

8.3 验证 loader 是否识别 Monarch tensor

编译:

```
cd ~/monarch_llama/llama.cpp-monarch
cmake --build build -j
```

运行:

```
./build/bin/llama-cli \
-m ~/monarch_llama/gguf_outputs/llama2-7b-monarch-extra.gguf \
-p "Hello" \
-n 8 \
2>&1 | tee ~/monarch_llama/logs/step3_monarch_loader.log
```

检查 loader 日志:

```
grep "monarch: layer" ~/monarch_llama/logs/step3_monarch_loader.log | wc -l
```

应出现类似 monarch: layer ... enabled。完整数量必须以本次参数包和目标 projection 清单为准。

Qwen2 loader 位于 src/models/qwen2.cpp, 当前只实现了 Q/O:

```
create_tensor_monarch(layer.wq_monarch, i, LLM_TENSOR_ATTN_Q, "attn_q", layer.wq);
create_tensor_monarch(layer.wo_monarch, i, LLM_TENSOR_ATTN_OUT, "attn_output", layer.wo);
```

因此 Qwen2 的矩形 K/V factor 会被 strict loader 拒绝, 而不会被忽略或错误认领。

9. 修改 src/models/llama.cpp : graph 阶段

graph 在 factor 存在且满足约束时执行后续的 L/R/perm 优化路径；如果没有 factor、存在 weight scale 或加载 LoRA 时，仍回退到正常的 dense 路径。

9.1 当前 dense graph 位置

在 src/models/llama.cpp 的 graph 构建中，当前 Q/K/V 计算为：

```
auto [Qcur, Kcur, Vcur] = build_qkv(model.layers[il], cur,
    n_embd_head, n_head, n_head_kv, il);
```

attention output 计算为：

```
cur = build_attn(inp_attn,
    model.layers[il].wo, model.layers[il].wo_b, model.layers[il].wo_s,
    Qcur, Kcur, Vcur, nullptr, nullptr, nullptr, kq_scale, il);
```

9.2 NumPy 数值参考

为避免在修改 C++/GGML graph 时同时猜测 tensor 布局与 permutation 方向，仓库增加了独立的 NumPy 参考实现：

- llama.cpp-monarch/monarch_reference.py 中的 monarch_linear_reference() 按拟合脚本的实际顺序执行 $\text{blockdiag}(R) \rightarrow \text{gather}(\text{perm}) \rightarrow \text{blockdiag}(L)$ ；
- 同文件的 materialize_monarch_dense_reference() 通过显式构造 R/L 块对角矩阵和 permutation matrix 得到等价稠密矩阵，未复用直接 forward 的计算路径；
- test_monarch_reference.py 比较两条独立路径，并检查单块退化、gather 方向、多维输入、最小 float32 计算，以及错误宽度、非浮点和 NaN/Inf 输入。

本地复现命令：

```
python -m unittest discover -s llama.cpp-monarch/tests -p "test_monarch*.py" -v
```

9.3 当前公共 graph 入口

当前实现没有在 graph 中展开 dense 权重，而是通过 build_monarch_mm() 选择执行路径：

factor 未启用、存在 weight scale 或存在 LoRA
→ 原 build_lora_mm() dense/scale/LoRA 路径
factor 启用且没有上述冲突
→ llama_monarch_linear(input, L, R, perm)

projection 编号 0/1/2/3 分别对应 Q/K/V/O。输入若不连续，graph 会先创建 contiguous tensor。

9.4 Q/K/V 的优化实现

公共 build_qkv() 对分离的 Q/K/V 分别调用 build_monarch_mm()，然后执行上游 bias、clamp 和 reshape 逻辑。由 loader 的 enabled 状态决定是否真正进入 Monarch：

- LLaMA 可对 q/k/v 中符合方阵条件且具有完整 factor 的项启用；
- Qwen2 当前只会为 Q 设置 factor，K/V 因 GQA 矩形 shape 保持 dense；
- Qwen2 的 Q/K/V bias 沿用原 graph，Q bias 位于 Monarch Q 输出之后，只添加一次；
- fused QKV 架构仍走原 dense 路径。

9.5 O projection 的优化实现

LLaMA 与 Qwen2 在 O factor 启用且不存在 scale/LoRA 时，先让 build_attn() 返回未做 O projection 的 attention context，再调用 build_monarch_mm(..., projection=3)；否则按上游方式让 build_attn() 执行 dense O。若架构声明 O bias，则在 Monarch O 后再添加。

10. ggml 层修改

出于通用性，我们通过 GGML 通用 custom op 接口接入，尚不是面向产品性能的专用 backend。

10.1 通用 custom OP

我们通过 src/llama-monarch.cpp 使用 ggml_custom_4d() 构造 GGML_OP_CUSTOM，输入为 input/L/R/perm，输出 shape 与 input 相同。当前支持：

input: 连续 F32
L/R: 连续 F32 或 F16
perm: 连续 I32
计算: $\text{blockdiag}(R) \rightarrow \text{gather}(\text{perm}) \rightarrow \text{blockdiag}(L)$

10.2 已修改文件

相关文件对应修改如下:

```
src/llama-monarch.h/.cpp    # custom OP 构造和 portable CPU 计算  
src/llama-graph.h/.cpp      # graph 选择、dense/LoRA/scale 回退  
src/CMakeLists.txt          # 编译接入
```

10.3 CPU reference 实现

基于MONET优化后的block-wise计算逻辑实现并对应优化。单 token decode 的逻辑如下:

```
// tmp1 = x @ R_blockdiag  
for b in range(num_blocks):  
    for j in range(block_size):  
        tmp1[b * B + j] = 0  
        for i in range(block_size):  
            tmp1[b * B + j] += x[b * B + i] * R[b][i][j]  
  
// tmp2 = tmp1[perm]  
for j in range(hidden):  
    tmp2[j] = tmp1[perm[j]]  
  
// y = tmp2 @ L_blockdiag  
for b in range(num_blocks):  
    for j in range(block_size):  
        y[b * B + j] = 0  
        for i in range(block_size):  
            y[b * B + j] += tmp2[b * B + i] * L[b][i][j]
```


11. Monarch-only GGUF 规划

若不做进一步优化，则 `monarch-extra.gguf` 同时保存 `dense weight` 和 `Monarch tensor`，因此文件更大。

目标是：

```
llama2-7b-monarch-only.gguf
```

该文件删除被替换层的 `dense weight`，只保留：

```
blk.i.attn_q.monarch_l  
blk.i.attn_q.monarch_r  
blk.i.attn_q.monarch_perm
```

此时才会真正减少模型大小。因此需要进行如下改动：

11.1 Converter 修改

在 `convert_hf_to_gguf_monarch.py` 中增加参数：

```
--monarch-only
```

如果启用：

1. 对已存在 `Monarch` 参数的 `q/k/v/o` 层，不写原 `dense weight`；
2. 只写 `monarch_l` / `monarch_r` / `monarch_perm`；
3. 对没有 `Monarch` 参数的层，仍然写 `dense weight`。

11.2 Loader 修改

`Monarch-only` 模型中，部分 `dense tensor` 不存在。因此 `loader` 需要允许：

```
attn_q.weight 缺失，但 attn_q.monarch_l/r/perm 存在
```

也就是说：

```
if dense weight exists:
    load dense
if Monarch tensor exists:
    load Monarch
if both exist:
    allow fallback or debug
if neither exists:
    error
```

当前 `create_tensor_qkv(...)` 默认会强制加载 dense q/k/v。修改为：

```
q/k/v/o dense weight 可选
Monarch weight 可选
二者至少存在一种
```

12. Android 端部署修改

12.1 当前 Android 部署方式

基础 Android 部署路径为：

```
Windows / WSL 生成 GGUF
↓
Windows 使用 Android NDK 编译 llama.cpp
↓
adb push 可执行文件和 GGUF
↓
手机端运行 llama-cli / llama-bench
```

该流程与普通 llama.cpp Android 部署类似。

12.2 Android 端需要同步的源码修改

当 llama.cpp-monarch 修改完成后，Android 端不能再使用原始 llama.cpp-baseline 编译。

必须使用：

llama.cpp-monarch/

因为其中包含：

1. llama_layer 中的 Monarch tensor 字段
2. LLaMA loader 中的 Monarch tensor 创建逻辑
3. graph 中的 Monarch projection 逻辑
4. `GGML\_OP\_CUSTOM` 便携 CPU correctness reference
5. Windows/Android 共用的 graph 与回退逻辑

12.3 Android 编译命令

Windows PowerShell 中：

```
$monetNdk="C:\Users\user\AppData\Local\Android\Sdk\ndk\28.2.13676358"
```

```
cd C:\Users\user\Documents\GitHub\MONET-llama.cpp
```

```
cmake -S llama.cpp-monarch -B llama.cpp-monarch\build-monet-android-arm64 -G "Ninja" `
-DMAKE_BUILD_TYPE=Release `
-DBUILD_SHARED_LIBS=OFF `
-DGGML_OPENMP=OFF `
-DMAKE_TOOLCHAIN_FILE="$monetNdk\build\cmake\android.toolchain.cmake" `
-DANDROID_ABI=arm64-v8a `
-DANDROID_PLATFORM=android-28
```

```
cmake --build llama.cpp-monarch\build-monet-android-arm64 `
--target test-monarch-op test-monarch-model --parallel 2
```

若目录已由同一 toolchain 配置，可只执行最后的 build 命令。

12.4 为什么源码修改后必须重新编译 Android 版

如果只换模型文件，例如：

```
llama2-7b-q4_k_m.gguf
→ llama2-7b-monarch-extra.gguf
```

理论上只需要重新 push 模型。

但如果修改了以下内容：

```
src/llama-model.h  
src/llama-model.cpp  
src/models/llama.cpp  
ggml op  
ggml CPU backend
```

则必须重新编译 Android 版：

```
llama-cli  
llama-bench
```

否则手机端二进制不包含 Monarch 支持。

12.5 推送 Android 可执行文件

```
adb shell "mkdir -p /data/local/tmp/llama && chmod 777 /data/local/tmp/llama"  
  
adb push build-android\bin\llama-cli /data/local/tmp/llama/llama-cli  
adb push build-android\bin\llama-bench /data/local/tmp/llama/llama-bench  
  
adb shell "chmod +x /data/local/tmp/llama/llama-cli /data/local/tmp/llama/llama-bench"
```

12.6 推送模型文件

Debug 阶段推送：

```
adb push llama2-7b-monarch-extra.gguf /data/local/tmp/llama/
```

Monarch-only 阶段：

```
adb push llama2-7b-monarch-only.gguf /data/local/tmp/llama/
```

如果做量化对比，也推送：

```
adb push llama2-7b-q3_k_m.gguf /data/local/tmp/llama/  
adb push llama2-7b-q4_k_m.gguf /data/local/tmp/llama/  
adb push llama2-7b-q6_k_m.gguf /data/local/tmp/llama/
```

12.7 手机端运行

```
adb shell
```

```
cd /data/local/tmp/llama
```

```
./llama-cli \  
-m llama2-7b-monarch-extra.gguf \  
-p "The future of on-device large language models is" \  
-n 64 \  
-t 8
```

12.8 手机端 benchmark

```
./llama-bench \  
-m llama2-7b-monarch-extra.gguf
```

可以考虑以下的对照 benchmark：

```
./llama-bench -m llama2-7b-q3_k_m.gguf  
./llama-bench -m llama2-7b-q4_k_m.gguf  
./llama-bench -m llama2-7b-q6_k_m.gguf  
./llama-bench -m llama2-7b-monarch-dense-f16.gguf  
./llama-bench -m llama2-7b-monarch-extra.gguf  
./llama-bench -m llama2-7b-monarch-only.gguf
```